

Use Case 2

Exploratory Analysis and Visualization

Objective · Find the trends, categories, hotspots and arrest patterns in the data.

Prepared for	Chicago Police Department
Prepared by	Accenture — Python Full Stack Program
Data source	chicago_crime_dataset.csv + 4 reference files
Toolchain	Python · Pandas · NumPy · Matplotlib · Seaborn
Warehouse	SQLite3 @ chicago_crime.db
Generated	17 August 2026, 11:10

1. Loading the cleaned extract

Use Case 1 wrote a cleaned, feature-engineered copy of the crime extract to `data/processed/chicago_crime_clean.csv`. Every figure in this report is built from that single file, so the numbers here and the numbers in the warehouse cannot disagree.

CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(config.CLEAN_CRIME_CSV, parse_dates=['date'],
                 dtype={'iucr_code': str, 'fbi_code': str})

print(f"{len(df):,} crimes | {df['year'].min()}-{df['year'].max()} | "
      f"{df['primary_type'].nunique()} crime types | "
      f"{df['community_code'].nunique()} community areas")
```

OUTPUT

```
2,000 crimes | 2015-2023 | 16 crime types | 77 community areas
```

2. Crime trend over the years

What is being asked: Plot the total number of crimes per year and read the chart: is crime in Chicago rising or falling over the reporting period?

CODE

```

crimes_per_year = df.groupby('year').size()
print(crimes_per_year.to_string())

# Least-squares slope: the direction of travel in crimes per year.
years = crimes_per_year.index.to_numpy()
counts = crimes_per_year.to_numpy()
slope, intercept = np.polyfit(years, counts, 1)

first, last = counts[0], counts[-1]
print(f"\nMean per year    : {counts.mean():.1f}")
print(f"Std deviation      : {counts.std(ddof=1):.1f} "
      f"({counts.std(ddof=1) / counts.mean() * 100:.1f}% of the mean)")
print(f"Peak year          : {years[counts.argmax()]} ({counts.max()} crimes)")
print(f"Lowest year         : {years[counts.argmin()]} ({counts.min()} crimes)")
print(f"Change {years[0]}->{years[-1]} : {(last - first) / first * 100:+.1f}%")
print(f"Trend slope        : {slope:+.2f} crimes per year")

```

OUTPUT

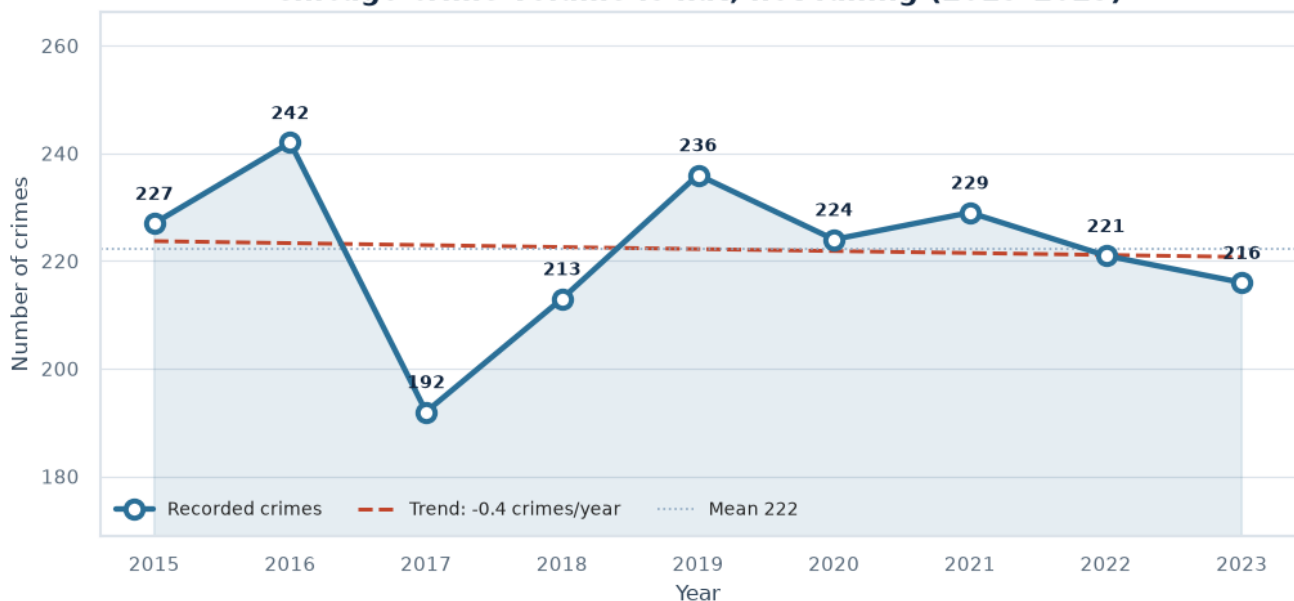
```

year
2015    227
2016    242
2017    192
2018    213
2019    236
2020    224
2021    229
2022    221
2023    216

Mean per year    : 222.2
Std deviation     : 14.5 (6.5% of the mean)
Peak year        : 2016 (242 crimes)
Lowest year       : 2017 (192 crimes)
Change 2015->2023 : -4.8%
Trend slope       : -0.37 crimes per year

```

Chicago crime volume is flat, not falling (2015-2023)



Total recorded crimes per year with the fitted linear trend.

Insight → **Neither rising nor falling — the series is flat.** The fitted trend is -0.37 crimes per year against a mean of 222, i.e. effectively zero, and the year-to-year spread is only 6.5% of the mean. 2016 is the busiest year (242) and 2017 the quietest (192), leaving the whole period 4.8% lower than it started — a difference well inside normal year-to-year variation. For the CPD the operational reading is that total volume is stable, so resourcing decisions should be driven by *where* and *when* crime happens rather than by any expectation that the overall caseload is shrinking.

3. Top 10 crime categories

What is being asked: Identify the top 10 crime categories by Primary Type, with both the count and the percentage each one represents.

CODE

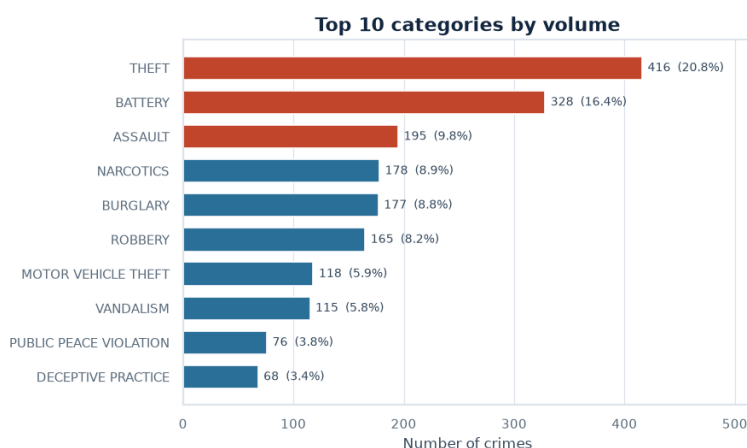
```
counts = df['primary_type'].value_counts()
top10 = (counts.head(10).rename('crimes').to_frame()
        .assign(pct_of_all=lambda t: (t['crimes'] / len(df) * 100).round(2),
                cumulative_pct=lambda t: t['pct_of_all'].cumsum().round(2)))
print(top10.to_string())

print(f"\nThe top 10 categories cover {top10['pct_of_all'].sum():.1f}% "
      f"of all {len(df):,} recorded crimes.")
print(f"The top 3 alone cover {top10['pct_of_all'].head(3).sum():.1f}%.")
```

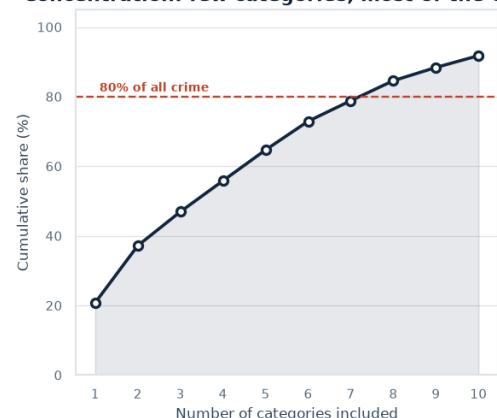
OUTPUT

primary_type	crimes	pct_of_all	cumulative_pct
THEFT	416	20.80	20.80
BATTERY	328	16.40	37.20
ASSAULT	195	9.75	46.95
NARCOTICS	178	8.90	55.85
BURGLARY	177	8.85	64.70
ROBBERY	165	8.25	72.95
MOTOR VEHICLE THEFT	118	5.90	78.85
VANDALISM	115	5.75	84.60
PUBLIC PEACE VIOLATION	76	3.80	88.40
DECEPTIVE PRACTICE	68	3.40	91.80

The top 10 categories cover 91.8% of all 2,000 recorded crimes.
The top 3 alone cover 47.0%.



Concentration: few categories, most of the crime



Top 10 crime categories by volume, with each category's share of the total.

Insight → Crime in this extract is heavily concentrated: **THEFT (20.8%), BATTERY (16.4%) and ASSAULT (9.75%) together account for 46.9% of every reported crime**, and the top ten of sixteen categories cover 91.8%. A prevention programme aimed at theft and battery alone would address 37% of the CPD's total caseload.

4. Arrests and crime outcomes

What is being asked: What percentage of recorded crimes actually result in an arrest, and is that rate steady across the years?

CODE

```
# arrest is stored 0/1, so the mean IS the arrest rate.
arrest_rate = df['arrest'].mean() * 100
print(f"Overall arrest rate : {arrest_rate:.2f}%")
print(f"Arrests          : {int(df['arrest'].sum()):,} of {len(df):,}")

by_year = df.groupby('year')['arrest'].agg(
    crimes='size', arrests='sum', arrest_rate=lambda s: round(s.mean() * 100, 2))
print()
print(by_year.to_string())

rates = by_year['arrest_rate']
print(f"\nSpread : {rates.min():.2f}% ({rates.idxmin()}) to "
      f"{rates.max():.2f}% ({rates.idxmax()}) - a {rates.max() - rates.min():.2f} pt range")
print(f"Std dev: {rates.std(ddof=1):.2f} pts")
```

OUTPUT

```
Overall arrest rate : 29.80%
Arrests          : 596 of 2,000
```

	crimes	arrests	arrest_rate
year			
2015	227	63	27.75
2016	242	62	25.62
2017	192	56	29.17
2018	213	72	33.80
2019	236	75	31.78
2020	224	68	30.36
2021	229	68	29.69
2022	221	72	32.58
2023	216	60	27.78

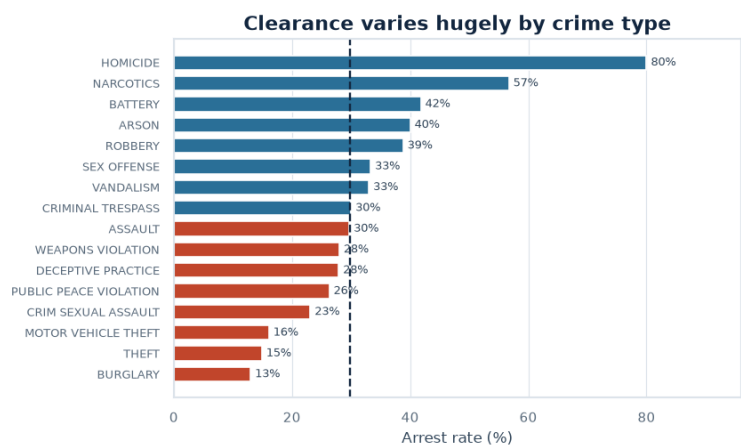
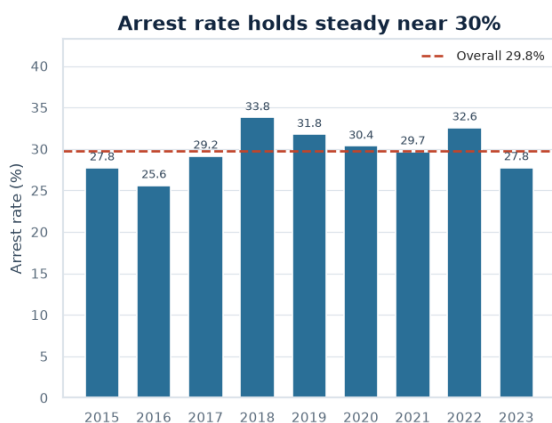
```
Spread : 25.62% (2016) to 33.80% (2018) - a 8.18 pt range
Std dev: 2.60 pts
```

CODE

```
# Which categories does the CPD actually clear?
by_type = (df.groupby('primary_type')['arrest']
           .agg(crimes='size', arrests='sum',
                arrest_rate=lambda s: round(s.mean() * 100, 1))
           .sort_values('arrest_rate', ascending=False))
print(by_type.to_string())
```

OUTPUT

primary_type	crimes	arrests	arrest_rate
HOMICIDE	10	8	80.0
NARCOTICS	178	101	56.7
BATTERY	328	137	41.8
ARSON	20	8	40.0
ROBBERY	165	64	38.8
SEX OFFENSE	21	7	33.3
VANDALISM	115	38	33.0
CRIMINAL TRESPASS	50	15	30.0
ASSAULT	195	58	29.7
WEAPONS VIOLATION	50	14	28.0
DECEPTIVE PRACTICE	68	19	27.9
PUBLIC PEACE VIOLATION	76	20	26.3
CRIM SEXUAL ASSAULT	13	3	23.1
MOTOR VEHICLE THEFT	118	19	16.1
THEFT	416	62	14.9
BURGLARY	177	23	13.0



Arrest rate per year (left) and per crime category (right).

Insight → **29.8% of recorded crimes end in an arrest, and that rate is stable** — it moves between 25.6% (2016) and 33.8% (2018) with a standard deviation of only 2.6 points, which is ordinary sampling noise on ~220 crimes a year rather than a real change in effectiveness. The variation that matters is by category, not by year: **HOMICIDE clears at 80.0% and NARCOTICS at 56.7%, while BURGLARY clears at 13.0% and THEFT at 14.9%.** Offences either witnessed by an officer or worked as a major investigation are cleared at four times the rate of property crimes reported after the fact — the strongest argument in this report for putting more officers on the street in the right places.

5. When does crime happen? Month vs day of week

What is being asked: Build a Seaborn heatmap of crime frequency pivoted by Month against DayOfWeek.

CODE

```

pivot = (df.pivot_table(index='month', columns='day_of_week',
                        values='case_number', aggfunc='count')
        .reindex(columns=DAY_ORDER))
pivot.index = [MONTHS[m - 1] for m in pivot.index]

print(pivot.to_string())
print(f"\nBusiest cell : {pivot.stack().idxmax()} with "
      f"{int(pivot.stack().max())} crimes")
print(f"Quietest cell: {pivot.stack().idxmin()} with "
      f"{int(pivot.stack().min())} crimes")

print("\nTotals by month:")
print(df['month'].value_counts().sort_index()
      .rename(index=lambda m: MONTHS[m - 1]).to_string())
print("\nTotals by day of week:")
print(df['day_of_week'].value_counts().reindex(DAY_ORDER).to_string())

```

OUTPUT

day_of_week	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
Jan	21	27	22	28	27	19	21
Feb	23	17	28	20	25	20	19
Mar	25	23	23	19	20	23	19
Apr	15	29	23	27	20	17	19
May	32	21	23	27	24	15	22
Jun	23	19	27	27	26	20	16
Jul	30	26	26	20	26	23	33
Aug	21	22	23	20	25	25	22
Sep	16	22	20	21	22	31	30
Oct	33	31	23	27	31	25	20
Nov	33	19	28	34	23	17	23
Dec	26	25	30	30	24	33	20

Busiest cell : ('Nov', 'Thursday') with 34 crimes
 Quietest cell: ('Apr', 'Monday') with 15 crimes

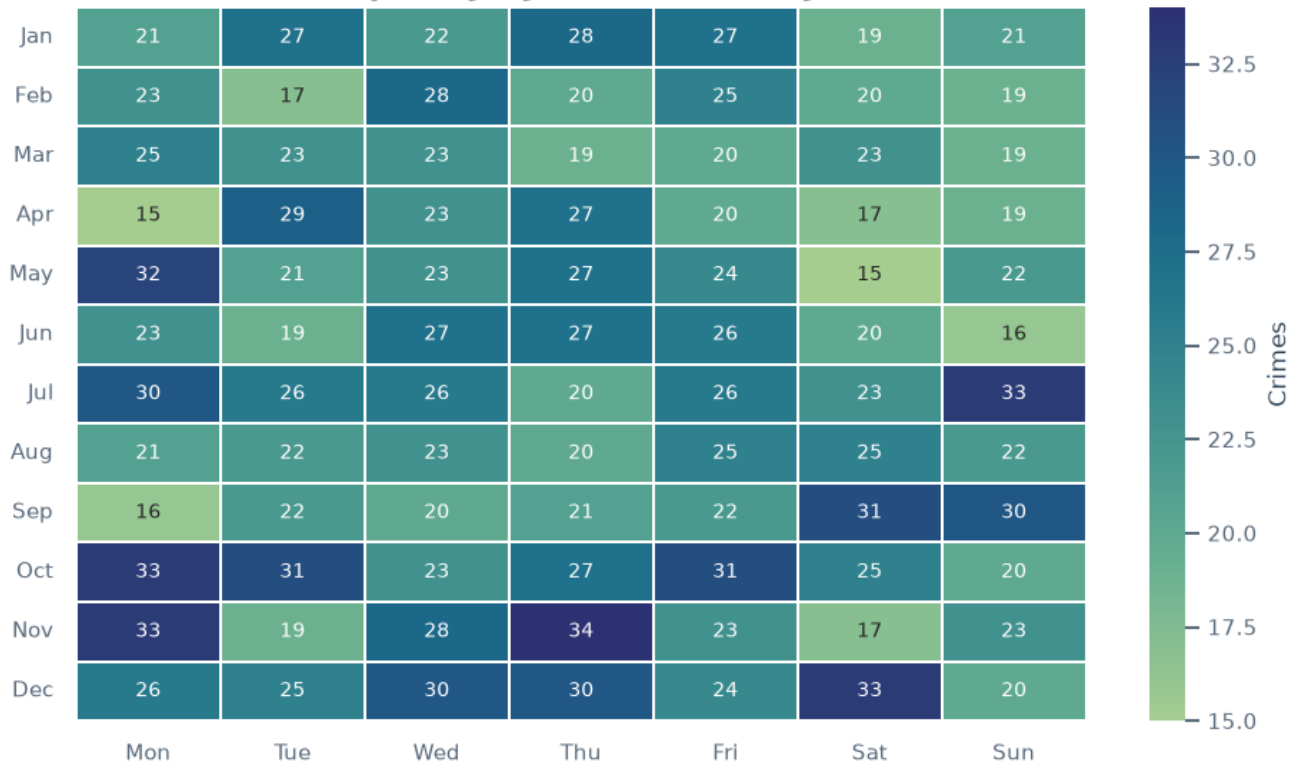
Totals by month:
 month

Jan	165
Feb	152
Mar	152
Apr	150
May	164
Jun	158
Jul	184
Aug	158
Sep	162
Oct	190
Nov	177
Dec	188

Totals by day of week:
 day_of_week

Monday	298
Tuesday	281
Wednesday	296
Thursday	300
Friday	293
Saturday	268
Sunday	264

Crime frequency by month and day of week



Crime frequency by month and day of week; darker cells are busier.

Insight → The seasonal signal here is real but modest, and it is **not** the textbook summer peak. **October is the busiest month (190 crimes), followed by December (188) and July (184); April is the quietest (150)** — a 27% gap between best and worst. Across the week the load is flatter still, and it leans the opposite way to the usual expectation: **Thursday is the heaviest day (300) and Sunday the lightest (264)**, so weekends are not the pressure point in this extract. The busiest single cell is November Thursdays (34 crimes) against a quietest of April Mondays (15). The planning implication is that a modest autumn/winter uplift is worth rostering for, while day-of-week rebalancing would buy the CPD very little.

6. Top community areas

What is being asked: List the ten community areas with the highest crime counts and plot them on a bar chart.

CODE

```
community = pd.read_csv(config.COMMUNITY_CSV)

top_areas = (df.groupby('community_code').size()
              .rename('crimes').reset_index()
              .merge(community[['community_code', 'community_name', 'population']],
                    on='community_code', how='left')
              .sort_values('crimes', ascending=False)
              .head(10).reset_index(drop=True))

# Raw counts favour big areas, so normalise by population too.
top_areas['crimes_per_10k'] = (top_areas['crimes'] /
                              top_areas['population'] * 10000).round(2)
top_areas['pct_of_all'] = (top_areas['crimes'] / len(df) * 100).round(2)

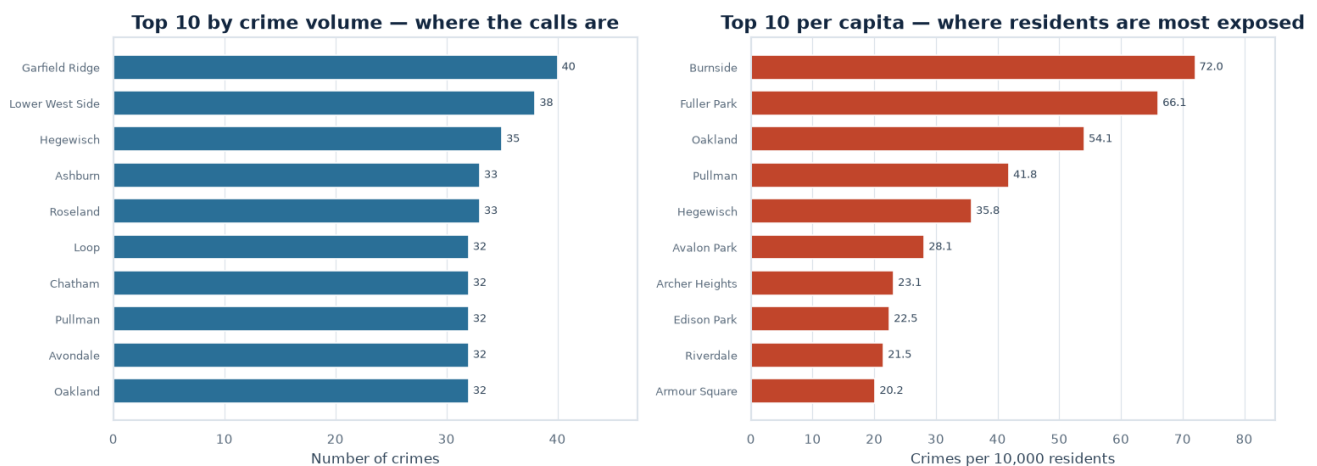
print(top_areas[['community_code', 'community_name', 'population',
                  'crimes', 'pct_of_all', 'crimes_per_10k']].to_string(index=False))

print(f"\nThese 10 of 77 community areas carry "
      f"{top_areas['pct_of_all'].sum():.1f}% of all crime.")
```

OUTPUT

community_code	community_name	population	crimes	pct_of_all	crimes_per_10k
56.0	Garfield Ridge	36445	40	2.00	10.98
31.0	Lower West Side	35769	38	1.90	10.62
55.0	Hegewisch	9781	35	1.75	35.78
70.0	Ashburn	42343	33	1.65	7.79
49.0	Roseland	43593	33	1.65	7.57
32.0	Loop	42298	32	1.60	7.57
44.0	Chatham	32408	32	1.60	9.87
50.0	Pullman	7648	32	1.60	41.84
21.0	Avondale	43083	32	1.60	7.43
36.0	Oakland	5918	32	1.60	54.07

These 10 of 77 community areas carry 16.9% of all crime.



Top 10 community areas by crime volume (left) and by crimes per 10,000 residents (right).

Insight → The ten worst community areas of 77 carry 16.9% of all recorded crime — but the two rankings tell different stories. Garfield Ridge tops the volume ranking with 40 crimes yet only 11.0 per 10,000 residents, while **Oakland records 32 crimes against a population of 5,918 — 54.1 per 10,000, five times Garfield Ridge's rate.** Ranking by raw volume points at large, populous areas; ranking per 10,000 residents promotes small areas whose residents face a far higher individual risk. Patrol allocation should follow the volume ranking (that is where the calls are), while prevention and community programmes should follow the per-capita ranking (that is where residents are most exposed).

7. Answers to the Use Case 2 questions

CODE

```
top_type = df['primary_type'].value_counts()
busiest_month = df['month'].value_counts().idxmax()
month_counts = df['month'].value_counts().sort_index()
yearly_rates = df.groupby('year')['arrest'].mean() * 100

print("Q1. Which crime category is most frequent?")
print(f"    {top_type.index[0]} — {top_type.iloc[0]:,} crimes "
      f"({top_type.iloc[0] / len(df) * 100:.2f}% of all reports)\n")

print("Q2. Is the arrest rate consistent across different years?")
print(f"    Yes. Overall {df['arrest'].mean() * 100:.2f}%; yearly rates run "
      f"{yearly_rates.min():.2f}%- {yearly_rates.max():.2f}% "
      f"(std dev {yearly_rates.std(ddof=1):.2f} pts).\n")

print("Q3. Which month has the highest crime frequency?")
print(f"    {MONTHS[busiest_month - 1]} — {month_counts[busiest_month]:,} crimes; "
      f"quietest is {MONTHS[month_counts.idxmin() - 1]} "
      f"({month_counts.min():,}), a {month_counts.max() / month_counts.min() - 1:+.0%} gap.")
```

OUTPUT

```
Q1. Which crime category is most frequent?
    THEFT — 416 crimes (20.80% of all reports)

Q2. Is the arrest rate consistent across different years?
    Yes. Overall 29.80%; yearly rates run 25.62%-33.80% (std dev 2.60 pts).

Q3. Which month has the highest crime frequency?
    Oct — 190 crimes; quietest is Apr (150), a +27% gap.
```

Insight → Three findings for the command team. **THEFT is the single most common crime** at one in five reports. **The arrest rate is consistent** at roughly 30% every year, so any improvement has to come from changing tactics rather than waiting for the trend. **October is the peak month**, 27% above April, with December close behind — the CPD's surge window is autumn into winter, not the summer that conventional wisdom would suggest.